QuillAudits

AUDIT REPORT

March 2026

For

Affordable Home

Table of Content

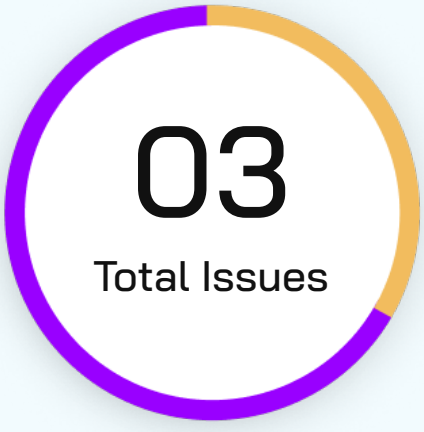
Executive Summary

Project Name	AHT
Protocol Type	RWA
Project URL	https://affordablehometoken.com
Overview	AHT and ReservationEscrow together form a tokenized home reservation protocol. AHT is a capped, burnable ERC20 that acts as the sole unit of account, with minting restricted by role. ReservationEscrow manages property listings, locking AHT into escrow when a user reserves — then either burning it permanently on fulfillment or refunding it on cancellation. The burn-on-fulfillment mechanic ties real-world housing activity directly to token supply, creating deflationary pressure with every completed reservation.
Audit Scope	The scope of this Audit was to analyze the AHT Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/xaviersoto/auditscope/tree/main
Branch	main
Contracts in Scope	AHT.sol ReservationEscrow.sol IAHT.sol IReservationEscrow.sol
Commit Hash	3b4846ebcc59975738df54f746bca24a9d2ed872
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	5th March 2026 - 6th March 2026
Updated Code Received	9th March 2026
Review 2	10th March 2026 - 11th March 2026
Fixed In	c80a858468026c2bb3e87e835c2f2c2f7213baf0

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	1 (33.3%)
Informational	2 (66.7%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	**1**	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Canceled Listing Cannot Be Re-Reserved Due to Stale reserver State	Low	Resolved
2	Floating pragma may introduce compilation inconsistencies	Informational	Resolved
3	Listing expiration timestamp is not validated during listing creation	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Low Severity Issues

Canceled Listing Cannot Be Re-Reserved Due to Stale reserver State

Resolved

Path

ReservationEscrow.sol

Function Name

`cancel()`

Description

When a reservation is canceled, listing.reserver is not reset to address(0) and listing.status is set permanently to Status.CANCELED. There is no mechanism to transition a listing back to Status.LISTED after cancellation. This means the admin or builder cannot re-open the listing for a new reserver even if the cancellation was intentional .

Impact: Low

Any canceled listing is permanently dead. The admin must deploy a new listing under a different id.

Likelihood: Medium

POC

1. Admin calls list(1, builder, price, deadline)
2. User calls reserve(1, ...) → status = RESERVED
3. Builder calls cancel(1) → status = CANCELED, reserver = userAddress (not cleared)
4. Admin attempts to call updateListing(1, ...) → reverts WrongStatus
5. No function exists to reset the listing back to LISTED
6. Listing id 1 is permanently unusable

Recommendation

modify cancel() itself to reset listing.reserver = address(0)

Informational Issues

Floating pragma may introduce compilation inconsistencies

Resolved

Path

ReservationEscrow.sol , AHT.sol

Description

The contract uses pragma solidity ^0.8.22, a floating pragma that permits compilation with any 0.8.x compiler version at or above 0.8.22. This means the deployed bytecode version is not pinned, and the contract could inadvertently be compiled with a future compiler version that introduces breaking changes or differing optimizations.

Impact: Low

Likelihood: Low

Recommendation

Lock the pragma to a specific compiler version, e.g. pragma solidity 0.8.22;.

Listing expiration timestamp is not validated during listing creation

Resolved

Path

ReservationEscrow.sol

Function Name

`list(), updateListing()`

Description

Neither list() nor updateListing() validate that reserveUntil is strictly in the future when it is non-zero. An admin or builder can set reserveUntil to a timestamp already in the past, which would cause reserve() to immediately revert with Expired for any caller — effectively listing a property that can never be reserved.

Impact: Low

Likelihood: Low

Recommendation

Add validation to ensure the expiration timestamp is either zero (no expiry) or in the future.

Functional Tests

- `list()` correctly creates a new property listing when the ID is unused, the builder address is valid, and the caller has the admin role.
- `updateListing()` successfully updates the listing price and expiration when called by the builder or an admin while the listing is in `LISTED` status.
- `reserve()` correctly escrows AHT tokens and marks the listing as `RESERVED` when the user provides the exact price and the listing is active.
- `reserve()` supports gasless approval through EIP-2612 permit when a valid permit signature is provided.
- `fulfill()` successfully burns the escrowed AHT tokens and marks the listing as `FULFILLED` when called by the builder or admin.
- `cancel()` correctly refunds escrowed AHT tokens to the reserver and updates the listing status to `CANCELED`.
- `reserve()` properly rejects reservations when the listing has expired.
- `reserve()` ensures only one reservation can exist for a listing at a time.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20.sol	Library / Base Contract	Core token logic (balances, transfers, allowances) in AHT	Correct implementation of ERC20 invariants	N/A
ERC20Capped.sol	Library / Extension	Enforces hard supply cap on AHT via _update() override	Cap logic correctly prevents mints beyond cap_	Misconfigured cap at deployment is permanent
ERC20Burnable.sol	Library / Extension	Exposes burn() and burnFrom() on AHT; called by ReservationEscrow on fulfillment	Burn correctly reduces totalSupply and reopens cap headroom	N/A
ERC20Permit.sol	Library / Extension	Enables EIP-2612 gasless approvals in both AHT & ReservationEscrow	Signature validation is correct and replay-safe	Malformed permit params revert rather than silently skip

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
AccessControl.sol	Library / Base Contract	Role-based gating for MINTER_ROLE, BUILDER_ROLE, and DEFAULT_ADMIN_ROLE	Role assignments are correctly initialised and managed post-deploy	Admin key compromise grants unlimited mint and operational control
ReentrancyGuard.sol	Library / Base Contract	Protects reserve(), fulfill(), and cancel() in ReservationEscrow	Guard correctly prevents cross-function reentrancy	N/A
Pausable.sol	Library / Base Contract	Allows admin to halt all AHT transfers in emergencies	Admin will invoke pause only in legitimate emergencies	Malicious or compromised admin can freeze all token movement
IERC20.sol	Interface	ReservationEscrow calls transferFrom and transfer on AHT via this interface	Token returns bool and does not silently fail	Non-compliant token would bypass TransferFailed guard
Solidity Compiler	Tool	Compilation of both contracts	Compiler has no known critical bugs for ^0.8.22	Floating pragma allows unintended compiler version at deployment

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
AHT.sol	constructor()	**What this function can do** Grant DEFAULT_ADMIN_ROLE and MINTER_ROLE to admin_, set token name, symbol, cap, and EIP-2612 permit domain **What this function cannot/should not do** Be called again after deployment; grant roles to address(0) **Main invariant(s)** Cap is immutable after deployment; admin_ holds both roles from genesis **Access level** Implicit restriction (once at deployment)
AHT.sol	mint()	**What this function can do** Mint new tokens to any recipient up to the hard cap when the contract is not paused **What this function cannot/should not do** Mint when paused; mint beyond cap_; be called by accounts without MINTER_ROLE **Main invariant(s)** totalSupply() ⩽ cap() at all times **Access level** MINTER_ROLE + whenNotPaused

Contract Name	Function	Category
AHT.sol	burn() / burnFrom()	**What this function can do** Permanently destroy tokens from the caller's balance (or an approved balance), reducing totalSupply and reopening cap headroom **What this function cannot/should not do** Burn when paused; burn more than the holder's balance; burn without sufficient allowance (burnFrom) **Main invariant(s)** totalSupply decreases by exactly the burned amount; cap headroom increases by the same **Access level** Any token holder (burn); approved spender (burnFrom)
AHT.sol	permit()	**What this function can do** Set a spending allowance via an off-chain EIP-2612 signature, enabling gasless approvals **What this function cannot/should not do** Accept expired or replayed signatures; approve on behalf of an address that did not sign **Main invariant(s)** Nonce increments after each valid permit; signature cannot be reused **Access level** Anyone with a valid signed permit from the token owner

Contract Name	Function	Category
ReservationEscrow.sol	list()	**What this function can do** Create a new LISTED property entry with a unique ID, assigned builder, AHT price, and optional expiry **What this function cannot/should not do** Reuse an existing listing ID; accept address(0) as builder; be called by non-admin **Main invariant(s)** Every listing ID maps to exactly one unique listing; status is always LISTED on creation **Access level** DEFAULT_ADMIN_ROLE only
ReservationEscrow.sol	updateListing()	**What this function can do** Update the priceAHT and reserveUntil of an existing listing while it is still in LISTED status **What this function cannot/should not do** Update a RESERVED, FULFILLED, or CANCELED listing; be called by addresses that are neither the builder nor admin **Main invariant(s)** Only LISTED listings are mutable; status is never changed by this function **Access level** Listing builder or DEFAULT_ADMIN_ROLE

Contract Name	Function	Category
ReservationEscrow.sol	reserve()	**What this function can do** Escrow the exact AHT price into the contract, transition listing to RESERVED, and record the reserver's address (with optional EIP-2612 permit) **What this function cannot/should not do** Accept a mismatched amount; reserve an expired, already-reserved, or non-LISTED listing; re-enter **Main invariant(s)** escrowed[id] == listing.priceAHT after success; listing.reserver is always set to a non-zero address **Access level** Any external caller (nonReentrant)
ReservationEscrow.sol	fulfill()	**What this function can do** Permanently burn all escrowed AHT for a RESERVED listing and transition its status to FULFILLED **What this function cannot/should not do** Fulfill a non-RESERVED listing; be called by anyone other than the builder or admin; re-enter **Main invariant(s)** escrowed[id] == 0 after fulfillment; tokens are truly burned (not sent to 0xdead) **Access level** Listing builder or DEFAULT_ADMIN_ROLE (nonReentrant)

Contract Name	Function	Category
ReservationEscrow.sol	cancel()	**What this function can do** Refund the full escrowed AHT amount to the original reserver and transition the listing to CANCELED **What this function cannot/should not do** Cancel a non-RESERVED listing; send funds to address(0); be called by unauthorized callers; re-enter **Main invariant(s)** escrowed[id] == 0 after cancellation; reserver receives exactly the amount originally escrowed **Access level** Listing builder or DEFAULT_ADMIN_ROLE (nonReentrant)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
AHT.sol	ERC20	None — token contract only; no assets held
ReservationEscrow.sol	ERC20 (AHT)	Escrowed reservation deposits pending fulfill or cancel

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
AHT.sol	mint()	—	ERC20 (AHT)	MINTER_ ROLE	Uncapped minting if role is compromised; no per-tx limit
AHT.sol	burn()	ERC20 (AHT)	–	Token Holder	None — intentional supply reduction
AHT.sol	burnFrom()	ERC20 (AHT)	–	Approved Spender	Allowance abuse if approval is misconfigured
AHT.sol	permit()	–	Allowance	Signature Holder	Signature replay on chain forks; expired deadline not enforced by caller

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Reservation Escrow.sol	reserve()	ERC20 (AHT)	–	Public	Front-run risk; permit griefing via frontrunning the permit call
Reservation Escrow.sol	fulfill()	–	– (burn)	Builder / Admin	Burned permanently; no recovery if called in error
Reservation Escrow.sol	cancel()	–	ERC20 (AHT)	Builder / Admin	Refund goes to listing.reserver; stale reserver after cancel blocks re-listing
Reservation Escrow.sol	list()	–	–	Admin	No expiry-in-future validation; mis-set reserveUntil bricks the listing
Reservation Escrow.sol	updateListing()	–	–	Builder / Admin	No expiry-in-future validation; price can be set to 0, allowing free reservations

Closing Summary

In this report, we have considered the security of AHT Protocol. We performed our audit according to the procedure described above.

No critical issues in AHt token, just 1 low and 2 issues of Informational severity were found, which have been noted and resolved by AHT team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

March 2026

For

Affordable Home

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com